

Étude de Cas

Système Pick-to-Light IoT

GE Healthcare Buc | Ligne Pristina | Supermarché Industriel

Digitalisation du processus de picking par technologie IoT

-60% Réduction temps de picking	-90% Réduction erreurs de picking	~6€ Coût par bac (vs 200€ industriel)	96% Économie vs solutions commerciales
---	---	---	--

1. Contexte et Problématique

GE Healthcare Buc fabrique des équipements médicaux d'imagerie (Senographe Pristina) sur la ligne Pristina. Le supermarché interne stocke environ 200 références de pièces détachées et consommables. Les Water Spiders (opérateurs logistiques) ont pour mission de préparer les chariots de pièces destinés aux postes de montage.

Problème	Impact terrain
Perte de 2-5 min par préparation	
Erreurs de picking fréquentes	
Risque de sous/sur-prélèvement	
Impossibilité d'audit et de contrôle	
Turnover et fatigue accrus	

2. Solution Pick-to-Light IoT

La solution proposée repose sur un système Pick-to-Light (PTL) entièrement développé en interne, basé sur des composants IoT grand public et intégré à l'infrastructure RFID et Oracle déjà en place à GE Healthcare Buc.

Composants principaux :

- ESP32 DevKit V1 Type-C — microcontrôleur WiFi/MQTT — 1 unité par zone de 7 bacs
- WS2812B LED RGB — 1 LED par bac — vert = prendre ici, éteinte = terminé
- TM1637 Afficheur 7 segments 4 chiffres — affiche la quantité à prélever
- Bouton poussoir 12mm étanche RUNCCI-YUN — confirmation du pick par l'opérateur
- Broker MQTT Mosquitto — déjà installé sur le Raspberry Pi du projet RFID
- Flask/MySQL — déjà en place, aucune infrastructure supplémentaire requise

Flux de fonctionnement — 7 étapes :

#	Etape	Description
1	Scan badge RFID	Water Spider scanne le badge RFID du chariot (Pepper C1)
2	Identification mission	Flask app identifie la mission et les items a prelever depuis MySQL
3	Commande MQTT	Flask publie les commandes vers les ESP32 des zones concernees
4	Activation PTL	LEDs vertes s'allument + afficheurs TM1637 affichent les quantites
5	Prelevement	Water Spider preleve la piece indiquee dans la bonne quantite
6	Confirmation	WS appuie sur le bouton → LED s'eteint → MQTT confirm publie
7	Mise a jour base	Flask reçoit la confirmation → MySQL mis a jour → mission TERMINEE

3. Architecture Hardware par Zone (7 bacs)

Composant	Qté	Rôle	Connexion
ESP32 DevKit V1 Type-C	1	Contrôleur WiFi/MQTT central	GPIO21-27, GPIO5, GPIO4...
TM1637 4 digits 0.56"	7	Affichage quantité à prendre	CLK + DIO (2 GPIO par module)
WS2812B LED RGB	7	Indication visuelle bac actif	Data chaîné — 1 GPIO total
Bouton 12mm étanche	7	Confirmation pick opérateur	1 GPIO par bouton + GND
Résistance 330 Ohm	1	Protection signal WS2812B	Série sur data GPIO5
Résistance 10k Ohm	1	Pull-up GPIO34 (bouton 7)	GPIO34 vers 3.3V
Condensateur 100µF	1	Protection alimentation LEDs	Entre 5V et GND
Alimentation USB 5V/1A	1	Alimentation zone complète	Via port USB-C ESP32

Bilan GPIO ESP32 pour 7 bacs :

Périphérique	GPIO utilisés	Pins ESP32
7 × TM1637 (CLK + DIO)	14 GPIO	GPIO21,22 / 18,19 / 16,17 / 14,13 / 26,25 / 27,23 / 5,?

Périphérique	GPIO utilisés	Pins ESP32
7 × WS2812B chaîne	1 GPIO	GPIO5 (data unique pour toute la chaîne)
7 × Bouton	7 GPIO	GPIO4 / 15 / 2 / 33 / 32 / 23 / 34
TOTAL utilisé	22 GPIO	Sur 34 GPIO disponibles (64% utilisé)

4. Intégration avec le Système RFID Existant

Le système Pick-to-Light s'intègre nativement dans l'infrastructure RFID déjà déployée à GE Healthcare Buc. Aucun serveur supplémentaire, aucun nouveau broker — tout repose sur la même pile technologique.

Composant	Technologie	État actuel	Action
Broker MQTT	Mosquitto (Raspberry Pi)	Déjà installé et configuré	✓ Réutilisé
Serveur applicatif	Flask Python	Déjà en place	✓ Étendu
Base de données	MySQL rfid_buc	Tables existantes	✓ Étendu
Sync Oracle	sync_oracle.py	OFs Released GLPROD	✓ Réutilisé
Réseau	WiFi usine GE	Déjà disponible	✓ Réutilisé

Topics MQTT Pick-to-Light :

Action	Topic MQTT	Payload JSON	Direction
Activation bac	rfid/zone-A/bac-1	{"qty": 3, "color": "green"}	Flask → ESP32
Confirmation pick	rfid/zone-A/bac-1/confirm	{"picked": true}	ESP32 → Flask
Tout éteindre zone	rfid/zone-A/reset	{"reset": true}	Flask → ESP32

5. Budget et Retour sur Investissement

Composant	Qté	Prix unitaire	Total
ESP32 DevKit V1 Type-C	1	10.00 €	10.00 €
TM1637 Afficheur 7 segments	7	1.00 €	7.00 €
WS2812B LED RGB module	7	0.40 €	2.80 €
Boutons RUNCCI-YUN 12mm (pack 5)	2	7.99 €	15.98 €
Résistances + condensateur	—	—	2.00 €
Fils, connecteurs, PCB proto	—	—	3.00 €
TOTAL par zone (7 bacs)			~41 €

Comparaison avec solutions PTL industrielles :

Solution	Coût/bac	Coût/zone (7 bacs)	Remarque
Solution IoT proposée	~6 €/bac	41 €/zone	Développement interne, IoT
PTL industriel standard	150-300 €/bac	1 050-2 100 €/zone	Systèmes Zetes, SSI Schaefer
Économie réalisée	-96 à -98%	-1 000 à -2 060 €/zone	✓ ROI immédiat

~6€ Coût / bac	-60% Temps de picking	-90% Erreurs picking	< 1 mois ROI
-------------------	--------------------------	-------------------------	-----------------

6. Plan d'Implémentation

Phase	Calendrier	Activités
Phase 1	Semaines 1-2	• Commande matériel (Amazon/AliExpress) • Montage prototype 1 zone (7 bacs) • Développement firmware ESP32 • Tests unitaires sur banc
Phase 2	Semaine 3	• Déploiement test en supermarché • Validation avec Water Spiders réels • Ajustements firmware et application Flask • Mesure des gains de performance
Phase 3	Semaine 4	• Déploiement toutes zones supermarché • Intégration complète RFID + PTL • Tests end-to-end complets • Documentation technique
Phase 4	Mois 2	• Formation des opérateurs WS • Mise en production officielle • Monitoring et maintenance • Rapport de performance